SentinelOps

A practical end-to-end guide

Asset condition and maintenance decision support | 29 October 2026

1. What the app helps you do

SentinelOps turns uploaded equipment readings into a clear maintenance workflow: spot an unusual change, investigate it, prepare the work, and record what happened afterward. It is useful for maintenance engineers, planners, technicians and reliability teams.

Think of it as a shared decision workspace. It helps people prepare and document work; it does not control equipment, issue site permits or prove when a machine will fail.

2. Start with your readings

1. Open **Data setup** and upload a UTF-8 CSV, XLSX or XLS file.

2. Match the timestamp, equipment ID and numeric sensor columns. Map a failure flag only if the file contains valid labels.

3. Review the quality report, then select **Process and unlock**. Other tabs become available after successful processing.

4. Visit **Control tower** to see which machines need attention, then open their evidence or case workspace.

Nothing loads automatically. The same upload gate is enforced by the backend.

Minimum data: 20 usable unique rows overall, with at least 10 valid readings per equipment/sensor for condition scoring. Use one row per equipment timestamp and include units in sensor names, such as `vibration_mm_s`. Dates display in UTC; timestamps without a timezone are interpreted as UTC. Numeric epoch and Excel serial timestamps need conversion first.

Quality rules: headers must be unique and non-empty. Rows with invalid timestamps, missing equipment IDs or no usable selected sensors are removed; exact duplicate rows are removed too. More than 30% unreadable timestamps blocks processing. Failure labels, when selected, must all be 0/1 or true/false. Missing sensor readings are reported, not silently filled in the history.

3. Add the records behind maintenance decisions

Sensor readings alone cannot tell the app which spare is available or who can do the job. Add these tables in **Data setup**, using matching equipment IDs:

Record	What it adds
Assets	Location, owner, importance, required part, repair duration and skill
Parts	Stock, existing allocations, lead time and unit cost
Windows	Allowed work periods, skills and shared crew capacity
Orders	Imported maintenance history and recorded labour/material costs
Failures	Failure and restoration times for reliability calculations
Documents	Reference text, document title, revision and section
Dependencies / changes	Equipment relationships and operational change history

Upload tables separately as CSV, Excel or JSON, or use a complete JSON bundle containing readings and context. Import parts before an asset table that references them. Complete bundles validate together. Context files have a 10 MB limit; telemetry and complete bundles default to 50 MB.

The supplied synthetic CSV contains 25,920 readings for 12 pumps and 18 simulated failure onsets. It is suitable for demonstration. It does not include these separate operational tables, so empty parts, reliability or knowledge sections are expected.

Find your way around

4. What each tab is for

Tab	Use it to...
Data setup	Upload and check readings; add supporting records and see what is missing.
Control tower	See the fleet summary and the priority action list, including owners and blockers.
Asset intelligence	Find equipment, check its condition and review uploaded relationships and changes.
Condition & diagnostics	Explore sensor history and compare recent readings with that machine's baseline.
Alert triage	Investigate warnings, assign ownership, check readiness, approve proposals and record verification.
Maintenance planning	Create and track work orders; review approved work periods and crew commitments.
Parts readiness	Check free stock, hold parts for cases and see material shortages.
Reliability & impact	Review recorded repair times, operating exposure, availability and completed-work spending.
What-if analysis	Compare repair duration, parts, crew and production assumptions without changing saved work.
Maintenance knowledge	Search reference text you uploaded and check its revision and section.
ML & data assurance	Train a model on request and examine its test results and influential inputs.
Research & scope	Review supported business problems, current limits and the workspace audit history.

5. A normal working day

Find the issue. Start in Control tower. Open a flagged machine and review its sensor chart. Check whether the change could reflect operating conditions or a sensor problem before treating it as equipment damage.

Investigate it. In Alert triage, assign an accountable owner, set a review date and save findings. Choose a disposition: confirmed component issue, operating change, sensor issue or no fault found.

Prepare the work. For a confirmed component issue, supply the required part, quantity, repair duration and skill. Reserve the material and review the proposed work period. The app checks shared crew capacity and lists missing requirements. Its current material model supports one mapped part per asset case.

Approve the proposal. Review the work scope, production impact and permit prerequisites. Approval requires the necessary evidence, reserved material and a suitable crew window. It creates or updates a linked work order. This approval remains a planning record; site permission and physical work happen outside the app.

Verify and close. After the investigation or work, record a follow-up sensor reading, its time and comparable operating conditions. Verification must follow the source observation. Closing resolves related alerts and completes a linked order; it does not rewrite the uploaded readings or add a failure label. The app records evidence rather than independently certifying that a repair succeeded.

Keep a review copy. Export the case work pack for its evidence and decisions. Equipment and parts reviews can be exported as CSV; scenario comparisons as JSON. Reopen the case if further work is needed. Parts reservations remain until explicitly released; they are not stock-consumption records.

Read results with the right meaning

6. What the numbers mean

Condition status: the first 60% of each equipment/sensor history forms its baseline. The latest three valid readings are averaged and compared with that baseline. A change of at least 1.5 standard deviations is Watch; at least 3 is Critical, in either direction. Critical means an unusual change worth investigating, not confirmed damage.

Condition index: a 0-100 indicator based on the largest sensor shift. A lower value means a larger departure from the baseline. It is not a failure probability, a certified health score or remaining life. Charts show up to 200 representative points; calculations use the usable history.

Model results: training uses roughly the earlier 70% of distinct timestamps and tests on the later period. With failure labels, Random Forest classifies the recorded flags. Precision asks how many flagged rows were correct; recall asks how many labelled failure rows it found. The confusion matrix shows correct and incorrect classifications. Without labels, Isolation Forest reports unusual readings instead of accuracy.

Model training needs enough labelled examples in both periods. Perfect scores on synthetic data do not demonstrate real-world performance. The model does not establish future failure dates. Optional MLflow logging can be unavailable while model training still succeeds.

Reliability and spending: mean repair time uses recorded failure-to-restoration intervals. MTBF means operating hours divided by matched failure events. Recorded availability is operating hours divided by operating plus matched repair hours; it is not scheduled production availability. Spending uses completed imported orders with recorded labour and material costs. Costs use INR. Missing evidence stays unavailable.

What-if results: production shortfall is the positive gap between demand and standby-supported capacity, multiplied by work duration. Interruption cost uses the supplied hourly cost and duration. These are estimates under your assumptions, not proven savings, physical failure simulations or permission to carry out work.

7. Saved work and common questions

Situation	What to do or expect
Tabs remain locked	Finish column checking and processing; correct the displayed validation errors.
Site, owner or importance says unknown	Upload the assets table. Descriptive columns in telemetry do not populate the separate asset register.
Parts, knowledge or reliability is empty	Add stock, reference text, or failure/restoration and exposure records respectively.
Approval is blocked	Read the listed blockers: owner, findings, materials, duration, skill, window, crew or review checks.
Model training is refused	Check numeric sensors and chronological label coverage; do not invent labels just to enable training.
Connection error	Check the backend and the frontend's API routing, then retry.

Failed replacements preserve active data and saved work. A successful telemetry replacement or **Clear dataset** resets its associated context, workflows and model. Release reservations and reopen approvals before replacing context; changed context invalidates verification. Editing key investigation details clears old verification and review checks.

Saved data survives a process restart when SQLite storage is retained. Your browser stores the session identifier: another browser, private window or cleared browser storage may open a different workspace.

Run and look after the app

8. Local setup and architecture

The React/Vite interface sends `/api` requests to a FastAPI backend. Python performs validation, analytics and model training. SQLite stores active readings, results, context and workflow decisions. Pending, unprocessed uploads stay in memory.

Use Python 3.10+ and Node.js 18+; Node.js 20+ is recommended. From the repository root, run these commands in Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
cd backend
..\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

In a second terminal, starting at the repository root:

```
cd frontend
npm ci
npm run dev -- --host 127.0.0.1
```

Open `http://127.0.0.1:5173`. On macOS/Linux, use `.venv/bin/python`. FastAPI exposes interactive API documentation at `http://127.0.0.1:8000/docs`; API calls require the browser's `X-Session` header.

9. Hosting, storage and access

`npm run build` creates `frontend/dist`. A host must serve those files and route `/api` to FastAPI. The repository's Vercel configuration currently forwards API calls to a Render backend; confirm that destination for your environment. A GitHub push does not itself prove a successful deployment. Automatic deployment depends on the connected hosting settings.

Set `SENTINELOPS_DB` to a durable SQLite file path before starting the backend. Use a single app instance, back up the database consistently and test restoration. Ephemeral disks can lose data on redeployment. `SENTINELOPS_MAX_MB` changes the telemetry/bundle limit. Keep the same session context when checking restored workspace data.

The current browser session token is not a login or role-based security system. Add authentication and server-side authorization before sensitive multi-user use. Retention rules, quotas and upload scanning also need operational setup. Live equipment, CMMS, historian and supplier systems are not connected; the app does not send real notifications.

10. Checks and handover

Run backend tests with `python -m pytest -q` from `backend`, using the project environment. From `frontend`, install Chromium with `npx playwright install chromium`, then run `npm run e2e` and `npm run build`. Stop development servers first: browser tests start fresh servers on ports 8000/5173 and use an isolated temporary database. Set `SENTINELOPS_PYTHON` if the test runner needs a different Python interpreter.

The change report records **70 backend tests, 23 Chromium browser tests and a successful production build on 23 October 2026**. These are recorded implementation results, not a fresh live-service assessment. Legacy XLS fixtures, load testing and production hosting were not covered.

For field-by-field imports, use `docs/CONTEXT_SCHEMA.md`; for setup and analytics details, use `README.md`; for verification evidence, use `CHANGE_TEST_REPORT.md`. This guide reflects the local implementation at commit `2830983`, reviewed on 29 October 2026. Documentation only: no application changes, repository push or deployment was performed for this guide.